

OSSP

Skill Week 9

Roll No – 2520090191

Name - : N.Rohith

Task 1-:

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 5

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, 20, "Running");
    job_count++;
}

void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        if (waitpid(jobs[i].pid, NULL, WNOHANG) > 0)
            snprintf(jobs[i].state, 20, "Completed");
    }
}

void list_jobs() {
    update_jobs();

    printf("\nActive Jobs:\n");
    printf("Job ID\tPID\tStatus\n");

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state[0] != 'C') {
            printf("[%d]\t%d\t%s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].state);
        }
    }
}

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {
        sleep(5);
        return 0;
    }

    add_job(pid1);

    pid2 = fork();

    if (pid2 == 0) {
        sleep(8);
        return 0;
    }

    add_job(pid2);

    printf("Two background jobs started.\n");
    list_jobs();
}
```

```
GNU nano 8.7.1
Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, 20, "Running");
    job_count++;
}

void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        if (waitpid(jobs[i].pid, NULL, WNOHANG) > 0)
            snprintf(jobs[i].state, 20, "Completed");
    }
}

void list_jobs() {
    update_jobs();

    printf("\nActive Jobs:\n");
    printf("Job ID\tPID\tStatus\n");

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state[0] != 'C') {
            printf("[%d]\t%d\t%s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].state);
        }
    }
}

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {
        sleep(5);
        return 0;
    }

    add_job(pid1);

    pid2 = fork();

    if (pid2 == 0) {
        sleep(8);
        return 0;
    }

    add_job(pid2);

    printf("Two background jobs started.\n");

    list_jobs();

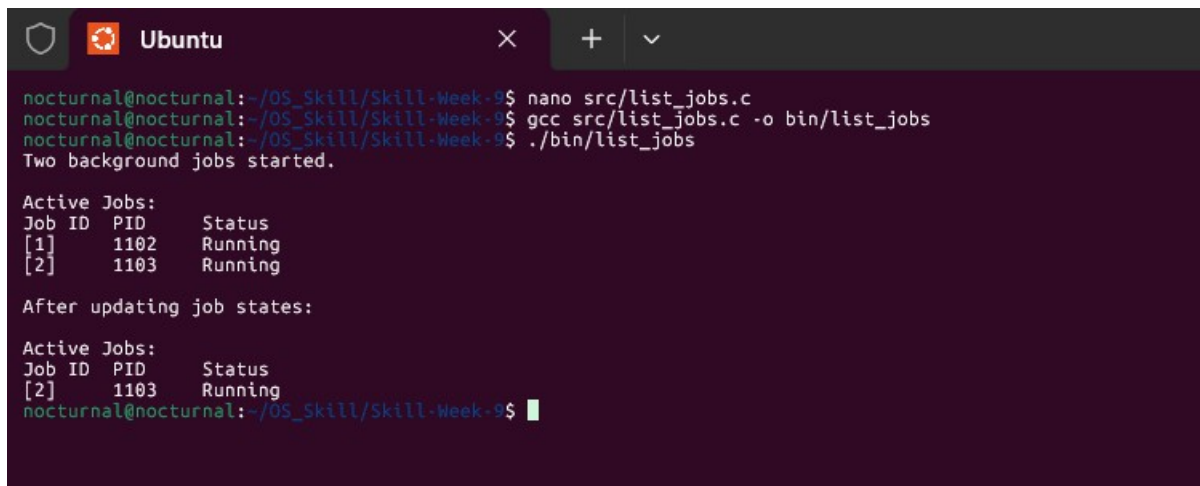
    sleep(6);

    printf("\nAfter updating job states:\n");
    list_jobs();

    waitpid(pid2, NULL, 0);

    return 0;
}
```

Output:-



The image shows a terminal window titled 'Ubuntu' with a dark background. The user 'nocturnal' is in the directory '~/OS_Skill/Skill-Week-9'. They execute three commands: 'nano src/list_jobs.c', 'gcc src/list_jobs.c -o bin/list_jobs', and './bin/list_jobs'. The output shows two background jobs started, followed by a table of active jobs. After updating job states, another table shows only job [2] as running.

```
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ nano src/list_jobs.c
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ gcc src/list_jobs.c -o bin/list_jobs
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ ./bin/list_jobs
Two background jobs started.

Active Jobs:
Job ID  PID    Status
[1]     1102   Running
[2]     1103   Running

After updating job states:

Active Jobs:
Job ID  PID    Status
[2]     1103   Running
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$
```

Task-2:

1. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    int status;

    printf("Starting background job...\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        printf("Child: Job started. PID = %d\n", getpid());

        raise(SIGSTOP);

        printf("Child: Job resumed in foreground.\n");

        sleep(3);

        printf("Child: Job completed.\n");

        return 0;
    }

    printf("Parent: Job PID = %d\n", pid);

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
        printf("Parent: Job state is Stopped\n");

    printf("\nSwitching job to foreground...\n");

    kill(pid, SIGCONT);

    printf("Parent: Job resumed.\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        printf("Parent: Job completed.\n");

    return 0;
}
```

Output -

```
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-$ nano src/foreground_switch.c
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-$ gcc src/foreground_switch.c -o bin/foreground_switch
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-$ ./bin/foreground_switch
Starting background job...
Parent: Job PID = 1201
Child: Job started. PID = 1201
Parent: Job state is Stopped

Switching job to foreground...
Parent: Job resumed.
Child: Job resumed in foreground.
Child: Job completed.
Parent: Job completed.
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-$ D
```

Task 3:-

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("SIGINT received!\n");
    else if (sig == SIGTERM)
        printf("SIGTERM received!\n");
    else if (sig == SIGUSR1)
        printf("SIGUSR1 received!\n");
}

int main() {
    sigset_t set;

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgid());

    printf("\nSignals are ready.\n");
    printf("Press Ctrl+C to test SIGINT.\n");

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);
    sigprocmask(SIG_BLOCK, &set, NULL);

    sleep(10);

    printf("SIGUSR1 is now unblocked.\n");

    sigprocmask(SIG_UNBLOCK, &set, NULL);

    printf("Program running...\n");

    while (1)
        sleep(1);

    return 0;
}
```

Output-:

```
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ nano src/signal_behavior.c
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ gcc src/signal_behavior.c -o bin/signal_behavior
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$ ./bin/signal_behavior
Process PID: 1276
Process Group ID: 1276

Signals are ready.
Press Ctrl+C to test SIGINT.

Blocking SIGUSR1 for 10 seconds...
^CSIGINT received!
SIGUSR1 is now unblocked.
Program running...
SIGUSR1 received!
SIGTERM received!
Killed ./bin/signal_behavior
nocturnal@nocturnal:~/OS_Skill/Skill-Week-9$
```

```
nocturnal@nocturnal:~$ kill -SIGUSR1 1276
nocturnal@nocturnal:~$ kill -SIGTERM 1276
nocturnal@nocturnal:~$ kill -9 1276
nocturnal@nocturnal:~$
```

Task 4-:

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
GNU nano 8.7.1 src/resume_job.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main() {
    Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: Job started. PID = %d\n", getpid());
        raise(SIGSTOP);

        printf("Child: Job resumed.\n");
        sleep(3);

        printf("Child: Job completed.\n");
        return 0;
    }

    job.job_id = 1;
    job.pid = pid;
    snprintf(job.state, 20, "Running");

    printf("Job table:\n");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        snprintf(job.state, 20, "Stopped");

        printf("\nJob stopped.\n");
        printf("[%d] PID=%d State=%s\n",
               job.job_id, job.pid, job.state);
    }

    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);

    snprintf(job.state, 20, "Running");

    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);
}
```

```
GNU nano 8.7.1 src/resume_job.c *
int main() {
    Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: Job started. PID = %d\n", getpid());
        raise(SIGSTOP);

        printf("Child: Job resumed.\n");
        sleep(3);

        printf("Child: Job completed.\n");
        return 0;
    }

    job.job_id = 1;
    job.pid = pid;
    snprintf(job.state, 20, "Running");
    printf("Job table:\n");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    waitpid(pid, &status, WUNTRACED);

    if (!WIFSTOPPED(status)) {
        snprintf(job.state, 20, "Stopped");

        printf("\nJob stopped.\n");
        printf("[%d] PID=%d State=%s\n",
               job.job_id, job.pid, job.state);
    }

    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);

    snprintf(job.state, 20, "Running");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    waitpid(pid, &status, 0);
    snprintf(job.state, 20, "Completed");
    printf("\nFinal job status:\n");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    return 0;
}
```

Output:-

```
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-9$ cd ~/OS_Skill/Skill-Week-9
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-9$ nano src/resume_job.c
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-9$ gcc src/resume_job.c -o bin/resume_job
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-9$ ./bin/resume_job
Job table:
[1] PID=1646 State=Running
Child: Job started. PID = 1646

Job stopped.
[1] PID=1646 State=Stopped

Sending SIGCONT...
[1] PID=1646 State=Running
Child: Job resumed.
Child: Job completed.

Final job status:
[1] PID=1646 State=Completed
nocturnal@nocturnal: ~/OS_Skill/Skill-Week-9$
```